

Workspace Models: Lightweight Robotic Memory via Saliency-Driven Supervision

Nitish Dashora^{a,§}, Douglas Chen^{b,§}, Idan Shenfeld^a, John Marangola^a,
Pulkit Agrawal^{a,†}, Max Simchowitz^{b,†}

[§]Project lead. [†]Equal advising.

^a Massachusetts Institute of Technology ^b Carnegie Mellon University

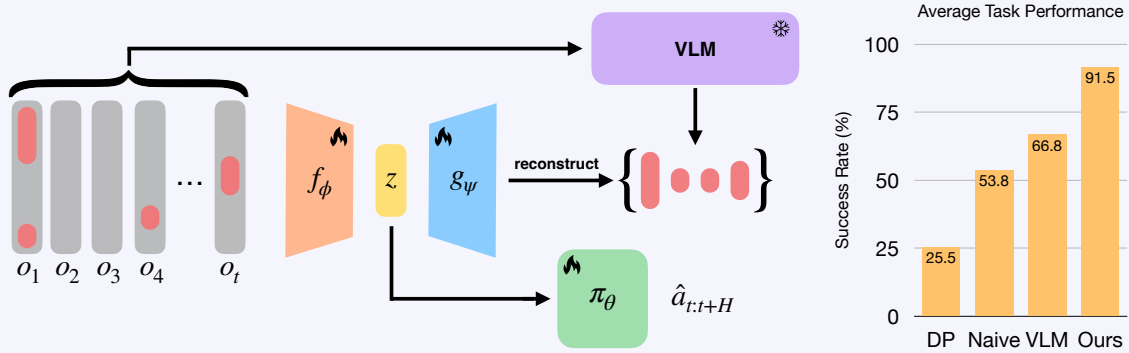


Figure 1: We propose the **Workspace Model**, an architecture trained to produce compressed latent representations of history used for downstream policy learning. Workspace Models are trained by encoding raw histories and reconstructing high-level salient information, as judged by a VLM, thus amortizing test-time compute. Across real and simulated tasks, our model solves memory-intensive tasks with low latency.

^a{dashora, idanshen, jmgola, pulkitag}@mit.edu

^b{dchen3, msimchow}@andrew.cmu.edu

Date: September 18, 2026

Abstract

Complex robotic manipulation tasks frequently require a long-term memory of past events and actions. As conditioning on full histories renders policies prone to spurious correlations and degrades performance, many approaches to policy memory involve compressing historical information through expensive VLM queries in-the-loop to process only task-salient information. In this paper, we propose an alternative approach in which computationally intensive VLM queries are made during train-time to learn a lightweight latent memory that can be efficiently queried at deployment time. Our representation, which we call the **workspace token**, is trained by (1) using a VLM to identify current and historical information necessary for completing a task, then (2) distilling these into the workspace token using a set-reconstruction decoder loss. In both simulation and hardware, we show that the workspace token can be used as a drop-in replacement for observations during deployment, enabling policies to solve memory-intensive tasks without the need for VLM reasoning in-the-loop. Interestingly, we found that workspace tokens are not only more lightweight but also lead to better policy performance.

1 Introduction

Solving hard decision-making problems requires memory of long temporal windows, as relevant information for control may be contained in prior observations [Kaelbling et al., 1998]. Despite the need for memory, today’s large-scale robot architectures are conditioned on either a single [Black et al., 2024, Zitkovich et al., 2023, Kim et al., 2024, Intelligence et al., 2025] or a short subsequence of past observations [Chi et al., 2023, Zhao et al., 2023, Shafiullah et al., 2022]. This is because naively increasing observation history can introduce generalization error during training [Kelly Hong, 2025] as irrelevant past frames make learned behaviors susceptible to latching onto spurious correlations with current actions and can also cause the model to encounter histories outside the policy’s training distribution [Geirhos et al., 2020, De Haan et al., 2019]. Distribution shift is worsened by compounding online error typical in imitation learning [Ross and Bagnell, 2010, Ross et al., 2011, Simchowicz et al., 2025]. These challenges are exacerbated in robotic applications, where data is considerably more limited and high dimensional than in natural language processing.

The challenges of memory have motivated numerous attempts to produce **summaries** of task history which contain what we term **salient information**: a limited amount of information from past frames needed to complete the task. Such compressed histories are still sufficient for task success, but limit extraneous or distracting information that may harm generalization. The most popular approach has been to deploy a powerful pre-trained model, such as a vision-language model (VLM), alongside the robot to provide explicit natural-language summaries of past events [Torne et al., 2026] or to select a limited number of relevant observation frames [Sridhar et al., 2025, Mark et al., 2026].

However, querying powerful models in robotics comes at a cost. Reactive policies must continually make queries, incurring high computational cost and suffering from latency that slows robot policies, introduces aliasing effects, and renders deployments more brittle. Moreover, imperfect summaries, e.g. those that lack key salient information, limit policy performance. This introduces a tradeoff where larger or more powerful models can provide more reliable summaries, but introduce greater latency at test-time, leading to the issues described above. For example, a medium-sized VLM [Team et al., 2025] can provide text-based summaries at lower latency, but text is limited in its ability to capture salient past details. Image-selection [Mark et al., 2026] can provide more information, but is more computationally demanding at inference time. Therefore, this work proposes an alternative solution:

Rather than using powerful models for deployment-time history curation, we can instead leverage them for **training-time supervision** of a lightweight, latent representation of memory.

Our Contributions. Concretely, we introduce **workspace models**: a lightweight memory encoder for robotic policies that achieves high performance in tasks that require long-term memory, without the deployment-time computation required by VLM-in-the-loop models.¹ Compared to these approaches, workspace models exploit a key asymmetry in robot learning: test-time compute scaling must stay limited at deployment and training data is hard to scale, but training-time compute scaling can be exploited. Workspace models take advantage by leveraging a new scaling axis we term **saliency-driven supervision**, where we expend inference compute on strong models to provide saliency labels at training time, which we use to train a compact encoder that captures all relevant historical information.

We compare workspace models to a number of representative baselines, including a VLM-based frame selection, similar to Mark et al. [2026] across simulated and real-world tasks that require long-term memory for counting, spatial recall, and test-time adaptation. Across these tasks, workspace mod-

¹Our approach is inspired by Global Workspace Theory in cognitive psychology, which posits that human beings maintain a bottleneck of information relevant to decision-making, memory, and perception [Bengio, 2019, Baars, 2005]

els enable effective memory with limited latency. Surprisingly, **workspace outperforms test-time key-frame selection**, due to a number of key design advantages that we expose through careful ablations. Ultimately, we view the workspace model as both

- Opening the door to **novel architecture design** for long-horizon memory
- A compelling proof of concept for **saliency-driven supervision** in domains where training data are scarce and/or computation is constrained at test-time

To this end, we stress that this manuscript’s implementation of a workspace model—using keyframe patches for saliency-driven supervision—serves as only a preliminary instantiation of the workspace principle, and look forward to future work which investigates other modalities of supervision, including text and video, as well as applying the workspace principle across domains.

2 Background: Training History-Based Robotic Policies

Preliminaries. We operate in the visual imitation learning setting. For a given task, we assume a dataset of N expert demonstration trajectories, $D = \{\tau_k\}_{k=1}^N$. Each trajectory is an ordered sequence of observations and actions $\tau_k = [(o_i, a_i)]_{i=1}^T$ with some episode length, T . Each observation contains the present image and proprioceptive state, $o_t = [I_t, x_t]$. The objective is to learn a receding horizon control policy $\pi(a_{t:t+H} | \text{history})$ where H is the action prediction horizon and history is some conditioning variable depending on $o_{1:t}$.

History-conditioned Policy Learning. Naively, one can select $\text{history} = o_{1:T}$ to be all past observations. However, prior work has identified a number of challenges with this approach. Long histories introduce the risk of overfitting since spurious correlates inside irrelevant historical information can be used to learn “shortcuts” [Geirhos et al., 2020] for action prediction that do not generalize to the online rollout distribution. In imitation learning, this can surface as “causal confusion” [De Haan et al., 2019] or “copycat behavior” [Wen et al., 2020] where the model exhibits poor performance by predicting actions from incorrect cues or simply copying previous actions, respectively.

To illustrate this intuition, we train state-based policies with varying history requirements to navigate to a point goal, $g \in \{-1, +1\}$ at a timestep L , starting from $x_0 = 0$; however, g is only shown to the agent once at some salient time $t_s \sim \mathcal{U}(1, L)$. Therefore, to succeed at reaching g for some L , the agent must have an observation history of size L or more. As seen in Figure 2, when training a policy conditioned on full history, the success rate degrades with increasing values of L , and even appears to saturate even as the number of demonstrations grows. As robot data is typically limited, the data-demands of naive history conditioning render the approach infeasible.

History Summarization for Robot Policy Learning. Here, we describe past approaches to mitigating the challenges associated with history-conditioning; we defer an extended related work to Appendix A.

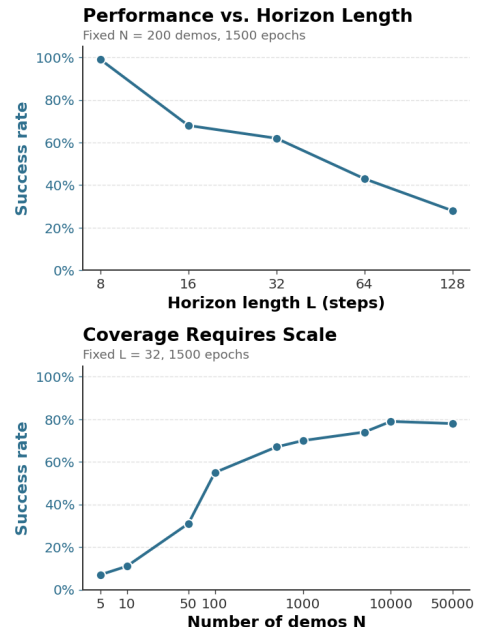


Figure 2: Performance degrades with POMDP horizon.

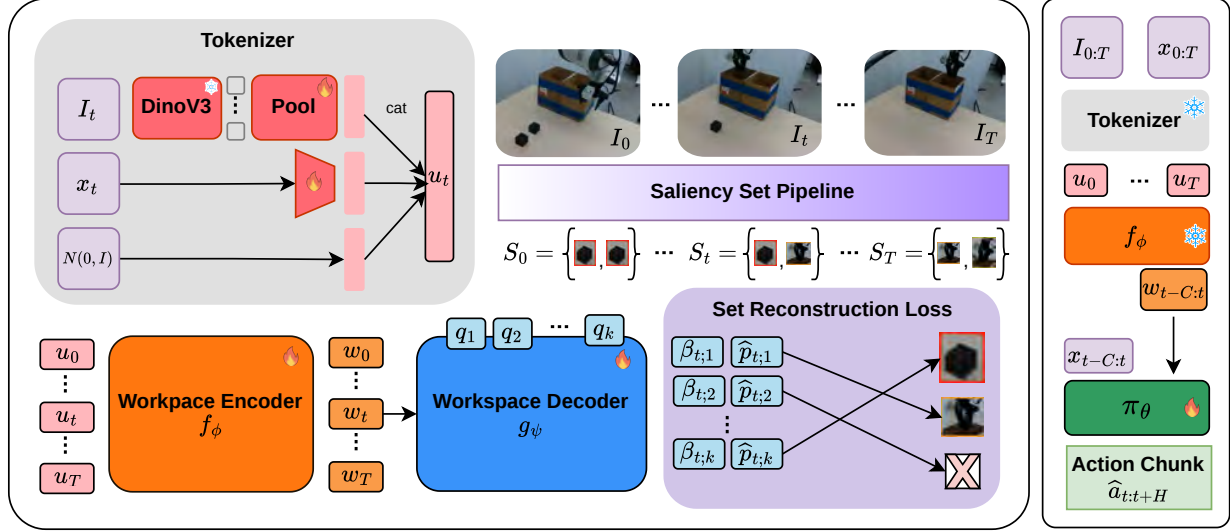


Figure 3: Workspace Model Training (left): Training involves tokenizing the image and proprioception through DinoV3 and a learned projection, respectively. These tokens are combined with an input slot at every timestep and fed to the workspace encoder to produce workspace tokens. Each workspace token, w_t , is supervised to reconstruct the set S_t through a Hungarian matching set reconstruction loss on the workspace decoder outputs. **Imitative Model Training (right):** Images and proprioception are fed into the tokenization and workspace pipeline to produce a short history of workspace tokens. These are used as input to train a Diffusion Policy π_θ , along with the raw proprioception, to output the future action chunks.

As mentioned in the Introduction, recent methods use a large model online to curate a task-relevant subset of observations $o_{t_1}, \dots, o_{t_k}$ or language summary l_t to feed into a downstream learner. For example, Torne et al. [2026] propose using a VLM to maintain a language memory and produce language subtasks for a downstream language-action model (VLA). Sridhar et al. [2025] similarly build a high-level policy for VLA subtask generation but maintain a visual keyframe memory through a VLM and find that text-only memory does not capture all information necessary for planning. Mark et al. [2026] directly use an API-queried VLM to select keyframes to persistently feed into a Diffusion Policy [Chi et al., 2023] and handle latency through masking recent frames, however still report latency-related failure modes. Crucially, all of these methods rely on using large-scale VLMs during inference-time for memory selection. Querying these large models takes time, which introduces significant latency during the robot tasks. This compromises policy reactivity, harming success rates on more dynamic tasks [Mark et al., 2026, Sridhar et al., 2025]. Moreover, large model queries incur significant financial cost, either because robots must ship with expensive GPUs, or pay for costly API queries whenever memory is needed. Finally, the summary generation system must consistently provide all sufficient information to prevent online catastrophic failures; this introduces a trade-off between summary quality and generation latency/computational expense.

3 Amortizing History Summarization via Workspace Models

In this work, we introduce the **workspace model** (Figure 3), a training pipeline and architecture that amortizes VLM queries during train-time to produce compact, latent summaries of salient history information. By design, our approach reduces the computational burden associated with querying powerful models at deployment time.

Workspace models train an autoregressive encoder $w_t = f_\phi(o_{1:t})$ to produce a latent summary we

call the **workspace token** w_t . During training, we use VLMs to curate a small **salient set** S_t of image patches from past and current visual observations which a VLM deems salient for determining the optimal action a_t (Section 3.2). Whereas prior work conditions on curated salient information at deployment, we instead train the encoder so that w_t encodes all the information in the salient set S_t . This is accomplished by co-training f_ϕ with a decoder $g_\psi(w_t)$ and penalizing the set-distance between the latter’s predictions and the ground truth S_t . Finally, we freeze the encoder f_ϕ , and learn a reactive, workspace-conditioned policy which produces actions from a short history of frozen workspace embeddings and proprioceptive states $\pi_\theta(a_{t:t+H} \mid w_{t:t-C}, x_{t:t-C})$. Via this approach, the reactive policy has access to a compact, latent history summary that contains all salient past information.

Representing Salient Information via Key-Patch Reconstruction. Given that salient events in manipulation tasks are often localized in space and time, we choose to represent the salient set S_t as an indexed set of DinoV3 [Siméoni et al., 2025] patch tokens from the images observed thus far. Concretely, $S_t = \{p_{t,i}\}_{i=1}^m$ where $p_{t,i}$ consists of the Dino patch token at location ℓ_i chosen from the image at time t_i . We note that the Dino patches already contain an encoding of their patch location ℓ_i . The saliency set is capped to be at most m patches. In Section 3.2 below, we describe how these patches are labeled and selected via prompted VLMs. We remark that this is only one possible representation of salient information, and leave exploration of other approaches to future work.

Workspace Encoder Architecture. We choose to parameterize the workspace encoder, f_ϕ as a transformer with causal masking that receives a sequence of input tokens, $[(\bar{o}_{t'}, z_{t'})]_{t' \leq t}$, where, $\bar{o}_{t'} = \text{tokenize}(o_{t'})$ are the Dino patch tokens of the observation in time t' . The $z_{t'}$ are inputs whose corresponding output-slots are the learned workspace tokens $w_{t'}$ (Appendix B), initialized to be $z_{t'} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \mathbf{I})$. Applying the encoder to this sequence, $f_\phi([(\bar{o}_{t'}, z_{t'})]_{t' \leq t})$, produces the sequence of workspace tokens $w_{1:t}$.

3.1 Workspace Model Training

Importantly, we represent salient information as a *set* S_t , which can be variable in size. Set structure necessitates specialized training and decoder design choices, which we describe here. We opt to train our decoder g_ψ following the DETR framework [Carion et al., 2020] for set reconstruction. Specifically, we parameterize our decoder g_ψ as a transformer to cross-attend learnable query tokens to a latent vector and produce m “slots” (recall: m is the maximum number of patches), each of which predicts information about a potential candidate set element together with a probability of whether that “slot” is occupied. Specifically, g_ψ outputs m pairs $g_\psi(w_t) = \{(\hat{p}_{t,j}, \beta_{t,j})\}_{j=1}^m$ consisting of (1) patch reconstructions $\hat{p}_{t,j}$ and (2) probabilities $\beta_{t,j}$ that the slot j is active at time t . The latter accounts for the salient set potentially containing fewer than m elements.

Encoder/Decoder Training. Training consists of two steps: a *matching step* that computes a mapping σ_t between available slots j and patches in S_t , and a *gradient step* on a reconstruction loss under that found matching. Given a binary $y \in \{0, 1\}$, let $\text{CrossEnt}(\beta; y) := \mathbf{I}\{y = 1\} \log(1/\beta) + \mathbf{I}\{y = 0\} \log(1/(1 - \beta))$ denote the standard cross-entropy loss. Moreover, we define a “null patch location” $i = 0$, to which we map unoccupied slots j , and define $y(i) := \mathbf{I}\{i \neq 0\}$, indicating that i is an active (non-null) position.

During the matching phase, we follow DETR and invoke the Hungarian algorithm [Kuhn, 1955] to obtain a map $\sigma : [m] \rightarrow \{0, 1, \dots, |S_t|\}$ between the m available slots and entries of S_t or null location 0. σ_t is injective into $|S_t|$ (i.e. never maps two slots j to the same $i \geq 1$, but can map many j to $i = 0$), and is computed by minimizing a cumulative matching cost $\sum_{\text{slots } j} c_t(j, \sigma_t(j))$, where

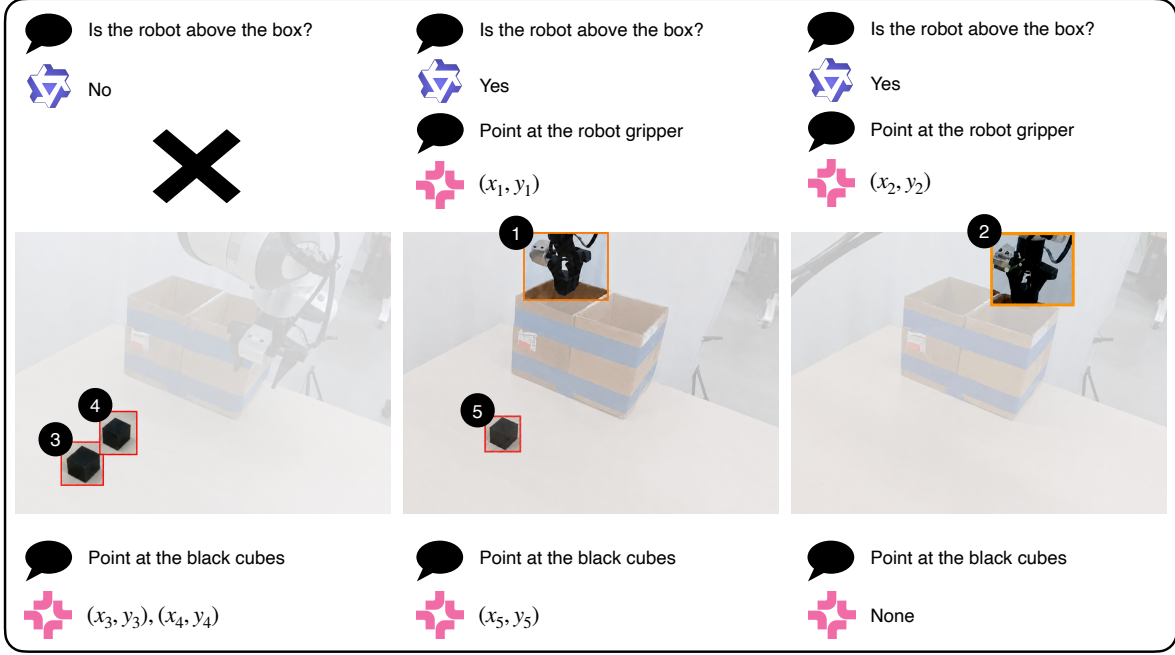


Figure 4: Our saliency-supervision pipeline which prompts Qwen (in purple) and MolmoPoint (in pink) to detect and locate events and build a growing memory. Orange boxes represent visual information which persists in the salient set over time. Red boxes represent “transient” information for that timestep only.

the cost $c_t(i, j) := y(i) \|p_{t;i} - \hat{p}_{t;j}\|^2 + \lambda_0 \text{CrossEnt}(\beta_{t;j}; y(i))$ penalizes mean squared error on non-null reconstructed patches and cross-entropy term on the indicators $y(i)$ that i is “active”.

Given this matching² σ_t , we compute a gradient with respect to a weighted combination of feature reconstruction (gated by slot j being active) and cross-entropy with the active labels $y(i)$:

$$\mathcal{L}_t^{\text{feat}} := \frac{1}{|S_t|} \sum_j y(\sigma_t(j)) \| \hat{p}_{t;j} - p_{t;\sigma_t(j)} \|^2, \quad \mathcal{L}_t^{\text{active}} = \frac{1}{m} \sum_{j=1}^m \text{CrossEnt}(\beta_{t;j}; y(\sigma_t(j))) \quad (3.1)$$

The full training objective is the sum of these per-timestep losses across the trajectory,

$$\mathcal{L} = \mathbb{E}_\tau \left[\sum_{t=1}^T (\lambda_1 \mathcal{L}_t^{\text{feat}} + \lambda_2 \mathcal{L}_t^{\text{active}}) \right] \quad (3.2)$$

We train this model through mini-batch SGD using the AdamW optimizer and a linear warmup cosine decay learning rate scheduler. The training parameters are listed in Table 4.

Workspace-Conditioned Policy. Having learned the workspace encoder f_ϕ , we train the workspace action-head π_θ to predict action-chunks [Zhao et al., 2023] conditioned on a short history of workspace tokens and proprioceptive states: $\pi_\theta(a_{t:t+H} \mid w_{t-C:t}, x_{t-C:t})$. We implement π_θ as a Diffusion Policy [Chi et al., 2023] further detailed in Appendix B.1.

3.2 VLM Saliency Labeling

We construct the salient set through a two-stage prompting depicted in Figure 4. First, we use the Qwen3-VL-8B-Instruct [Bai et al., 2025] model to identify key time steps $t' \leq t$ where salient events

²Importantly, we do not backpropagate gradients through the computation of σ_t .

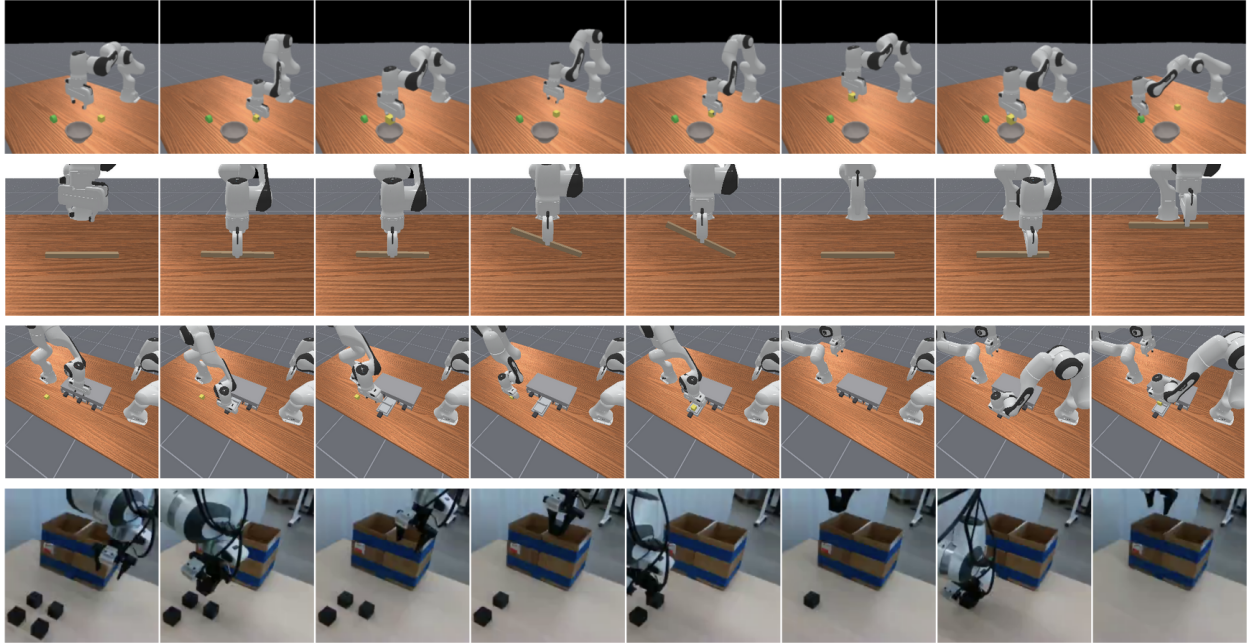


Figure 5: Every row depicts a representative demonstration for the tasks in order top-to-bottom: CUBEDROP, BALANCEBAR, DRAWERRECALL, and HALFANDHALF. The first three rows are selected 3rd-person camera renderings for the trajectory. The fourth row is the global camera used for training.

occur. Specifically, we provide Qwen with a set of classification questions to evaluate whether any given frame contains an event relevant to remember (e.g. is the salt put away?). To collect key-frames, we prompt Qwen with these questions on every frame of every trajectory. As a post-processing step, we filter out consecutive positive frames using a sliding window to remove redundant key-frames. The outcome is an ordered list of events to remember (e.g. salt was added to soup at time $t = 50$, knife was put in drawer at $t = 150$).

Subsequently, we use MolmoPoint [Clark et al., 2026], a VLM fine-tuned for open-vocabulary point-
ing, to extract salient patches at a regular frequency (refer to Appendix C for details) and for keyframes. At the regular frequency, MolmoPoint is given objects to persistently track (specified via prompt). For keyframes specifically, MolmoPoint is given a separate set of objects to track used only for keyframes. MolmoPoint then produces 2D coordinates for prompted objects for all keyframes and at the regular frequency. We collect these keyframe-points from $t' \leq t$ and any persistent object points at t , effectively making the salient set contain both what to remember and what to focus on now. We then convert these points (each associated with a time) to DinoV3 patches by simply choosing the unique patch in time and space which overlaps with the point. The union of these constitutes the salient set S_t . We attach example prompts in Appendix C.

4 Workspace Models Achieve High Task Success with Low Inference Latency

In this section, we compare workspace models to a number of history-conditioning baselines. We show, across simulated and real-world environments, that workspace models enjoy low control latency while strongly **outperforming natural baselines in task success**. In the remainder of the section, we describe the baselines and tasks, and expose the core design decisions that contribute to the strong performance of workspace models relative to alternatives.

Baselines. We compare workspace models, denoted **Wksp**, against three natural baselines: a vanilla Diffusion Policy, **VanillaDP**, which uses the current and previous frame as input; a history-conditioned Diffusion Policy, **HistoryDP** which conditions a sufficient history of frames by naively striding across previous observations (we find that full, unstrided history yields 0% success); and finally, **Keyframe**, inspired by [Mark et al. \[2026\]](#), which uses a VLM and a tuned prompt to select frames to add into Diffusion Policy context. We report baseline training and architecture details in [Appendix B.1](#). Importantly, **Wksp** uses very similar hyperparameters across all simulation and real-world environments. Baselines are tuned per-task to steelman their performance.

Simulation and Hardware Setups. We evaluate our methods against four memory-intensive tasks in both simulation and on real hardware. In simulation, we use ManiSkill3, built on SAPIEN [\[Tao et al., 2025, Chang et al., 2015, Mo et al., 2019, Xiang et al., 2020\]](#) to simulate our experiments on a Franka Research 3 robot [\[Fra, 2025\]](#). Further details are provided in [Appendix D](#). For real-world experiments, we use a Franka FR3 with an AgileX parallel jaw gripper with operational space control formulation [\[Khatib, 1987\]](#). Visual information is collected with a fixed Intel RealSense D435 global camera. Further details on teleoperation and controller information are provided in [Appendix D](#).

Table 1: Latency comparison across baselines.

Method	Latency (ms)	Relative
VanillaDP	41.3	1.00×
HistoryDP	41.3	1.00×
Keyframe	302	7.31×
Keyframe+Batch	199.5	4.83×
Wksp	94.9	2.3×
Wksp+Batch	52.9	1.28×
Wksp+Batch+KV	49.2	1.19×

4.1 Evaluation Tasks: Measuring Different Axes of Memory.

We test, in three simulated and one real task, three distinct memory-usage capabilities: **1)** counting under partial observability; **2)** spatial recall and **3)** in-context adaptation from past experience. We describe the tasks at a high level below, and defer details to [Appendix D](#). All baselines are trained via supervised behavior cloning from demonstrations.

To study the long-range counting capabilities, we design the CUBEDROP task in simulation, inspired by [\[Mark et al., 2026\]](#), in which a robot must place exactly 5 cubes into a bowl, where cubes leave the robot’s line-of-sight when correctly placed. We also evaluate on a hardware counterpart, HALFANDHALF, where the robot must place $K \in \{2, 4\}$ cubes evenly into two boxes that occlude the visual camera. Both CUBEDROP and HALFANDHALF necessitate memory, as partial observability obscures the running count from the current observation, while requiring that the memory does not interfere with precise execution (cube insertions).

Next, DRAWERRECALL tests spatial recall. Here, another robot first opens and puts a cube away into one of three drawers, then closes it, and a separate robot (our policy) must open the drawer containing the stored cube. Finally, we test in-context adaptation, in BALANCEBAR. Here a robot is given two attempts to lift a bar with an unknown center-of-mass (CoM) so that it remains level. The demonstrations start with lifting from the middle first, revealing the CoM, then grasping at the CoM, resulting in a level lift. The learned policy must learn to do the same: use history to determine the CoM, and reattempt at the correct grip position. We show demonstration trajectories in [Figure 5](#).

4.2 Workspace Models Successfully Amortize VLM Reasoning

Recall our central motivation: amortizing slow and computationally-intensive VLM-in-the-loop history summarization into a lightweight encoder that can be queried efficiently at deployment. Before evaluating downstream policy performance, we verify that this amortization improves run-time.

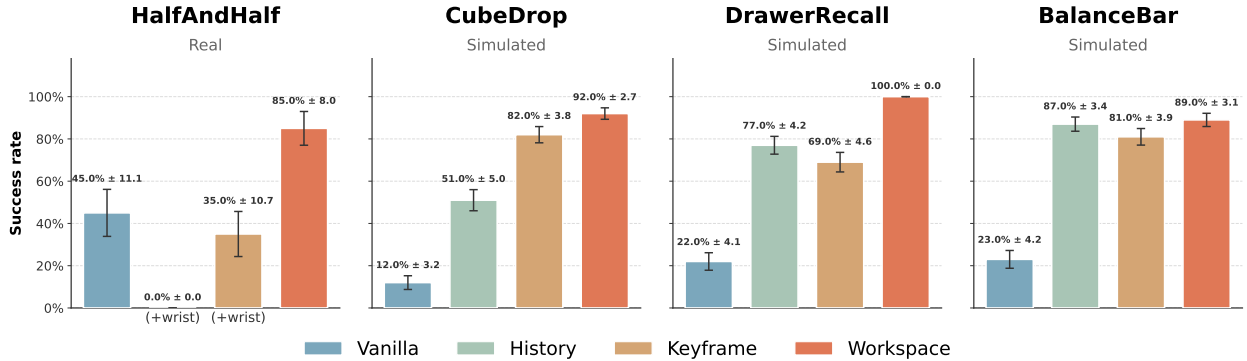


Figure 6: Success rates across 3 simulated (N=100) and 1 real task (N=20). For the real task, we use wrist cameras for the methods unable to pick the cubes correctly with a global camera only, denoted with “(+wrist)”

Specifically, we measure the average amount of time to run the policy across the previous action chunk and replan the next chunk (shown in Table 1). We measure relative to VanillaDP and find that **Wksp** takes ~ 1.2 times as long as a basic CNN perception stack as used in VanillaDP and HistoryDP. Meanwhile, **Keyframe** is ~ 5 times as costly, which would be significantly worse if using API call VLMs. We report numbers that leverage KV-caching and stacking observations between chunks into one batch. Together, these results confirm that workspace models achieve their core design goal - the expensive reasoning is done at train-time, leaving a lightweight and efficient encoder at deployment.

4.3 Workspace models surprisingly outperform all baselines.

Having established that workspace models successfully amortize VLM reasoning, we now evaluate whether this translates to downstream task performance. Figure 6 shows success rates across all four tasks. Workspace models achieve the highest success rate in every task, with a cross-task average of $91.5\% \pm 2.2$ significantly ahead of the next best baseline, **Keyframe**, at $66.8\% \pm 3.2$. Notably, this improvement holds in both simulation and the real world.

This is our most **surprising finding**: **Wksp** outperforms **Keyframe** not only on latency (which was the original motivation), but also on success rate. Below, we dig into the mechanisms that lead to this performance gap.

Table 2: Memory / Control failure rates across tasks.

Method	Bar (M/C)	Drawer (M/C)	Cube (M/C)
Wksp	0% / 100%	N/A	100% / 0%
VanillaDP	88% / 12%	71% / 19%	100% / 0%
HistoryDP	0% / 100%	50% / 50%	100% / 0%
Keyframe	0% / 100%	0% / 100%	100% / 0%

5 Why do Workspace Models Exhibit Better Task Performance?

In this section, we account for the surprising finding that workspace models, despite being targeted at *amortizing* key-frame lookup, actually **outperform** key-frame lookup.

5.1 Full Histories and Key-Frame History Both Induce Control-Failure

We first begin by categorizing the failure modes exhibited by different methods as either memory-related errors, where the robot correctly executes a skill but targets the wrong mode (i.e. putting 3 blocks away and pressing done instead of 5), or control-related errors, where the robot fails to execute a manipulation

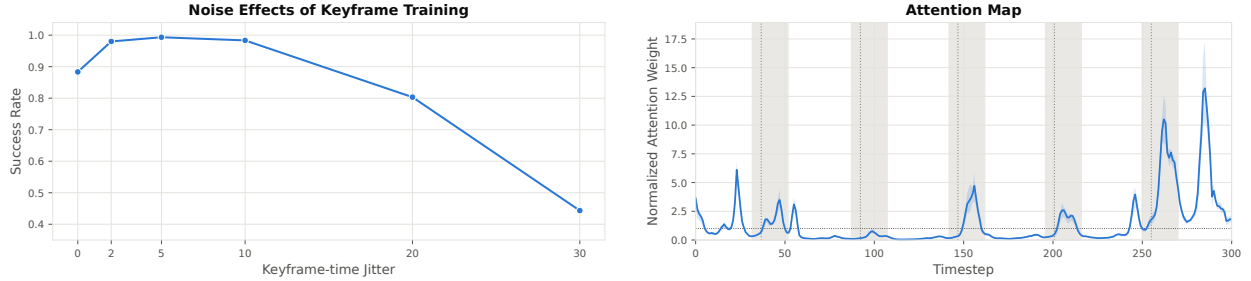


Figure 7: Left: Success as a function of noise from optimal timings. **Right:** Attention scores (average over 10 trajectories) measured relative to the average score at a given offset from the current timestep. Gray areas are segments of important events to remember

skill (i.e. missing a grasp of a block). We share results of failures within simulation experiments in Table 2, finding that VanillaDP unsurprisingly suffers from memory-related errors but rarely fails to execute an action mode.

Weaknesses of Frame Stacking Methods. The frame-stacking methods (i.e. HistoryDP and Keyframe) tend to contain good memory but fail more often with control. Wksp seems to be able to balance both memory and control. Qualitatively, HistoryDP and Keyframe select the correct task mode, but fail on fine-grained motions. In DRAWERRECALL for example, HistoryDP often selects the correct drawer to open, but misses the handle slightly. Keyframe exhibits these failure modes as well and sometimes freezes while opening the drawer. In real-world experiments, we observe that failures of control precision can resemble mode-selection failures unrelated to the correct memory: for example, Keyframe and HistoryDP both attempt to pick in the middle of the two cubes or to grasp 8-10cm above the cube. This led us to form a hypothesis that the performance gap in Figure 6 stems from a common failure mode across both HistoryDP and Keyframe - adding more frames into context introduces generalization error that outweighs the benefit of richer history.

5.2 Keyframe-based methods introduce control aliasing.

Next, we identify a second failure mode: explicit frame selection methods (whether strided, as in HistoryDP, or VLM-based, as in Keyframe) create an **aliased** input structure during training that may be prone to distribution shift from their narrow training distributions.

To examine this, we train Keyframe with oracle keyframe timings at both training and deployment and find imperfect performance. Interestingly, adding moderate jitter to the training distribution of event times significantly improves performance as shown in Figure 7, but then falls as this training neighborhood is expanded too wide. This illustrates the balance between making the distribution of keyframes wide enough to prevent learning a brittle policy and not so wide that history coverage becomes slim and causes overfitting, as shown by Mark et al. [2026]. The Keyframe method manages to introduce some jitter to counteract aliasing, as shown by its natural noise level ($\sigma \approx 10.1$) and strong performance with an oracle at test time (~ 0.96). However, its success then relies on noisy VLM curation during train-time and perfect curation at deployment. Next, we show how Wksp naturally remedies this by virtue of being a smooth representation.

5.3 Why do Workspace Models Generalize Better?

We now show that the workspace model exhibits smoothness in both its own training input, due to (1) training with full global attention over time, and (2) smoothness in its outputs (i.e. the policy inputs).

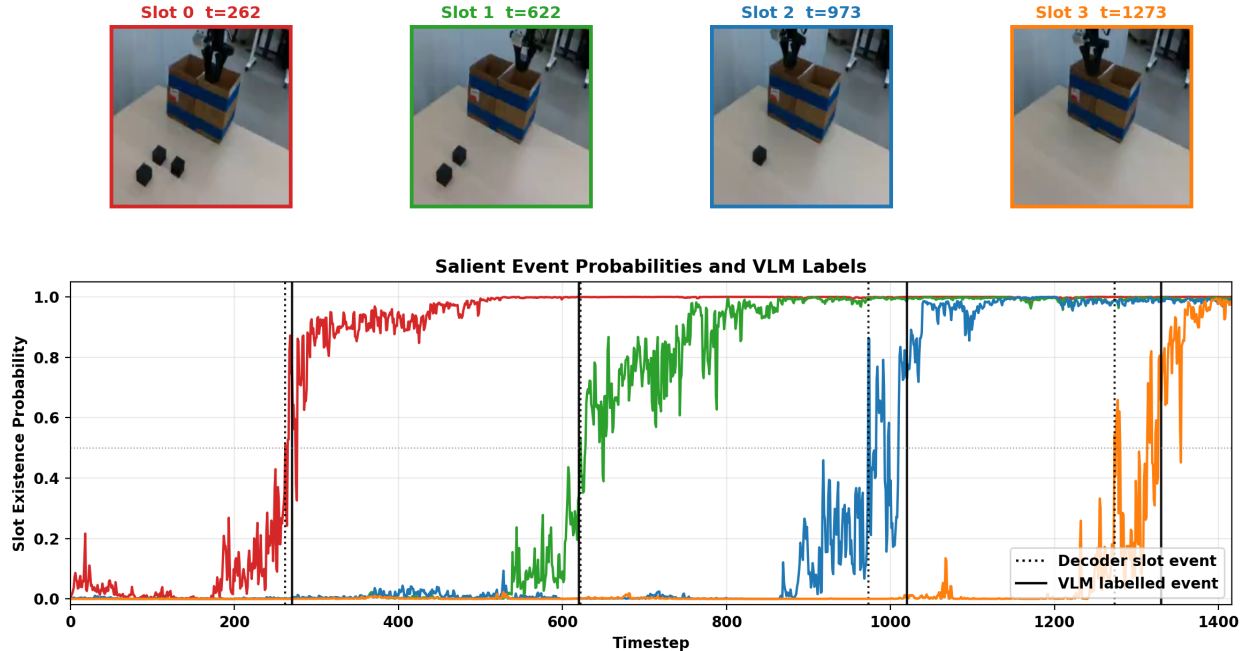


Figure 8: We show images and their timesteps which correspond to moments when a slot probability crosses a threshold of 0.5 (**top**). The moments where slots move above the threshold are “decoder slot events” shown with dotted lines. We also show the slot probabilities across time for every slot along with the VLM-supervised labels. As seen, decoder firings slightly diverge from the ground truth in order to match up to the event’s general visual features, creating a smoothing effect (**bottom**).

Benefits of global attention. As evidence of (1), we identify that the workspace encoder itself attends across all available timesteps, learning to attend smoothly to the temporal sequence as opposed to paying full attention to discrete timesteps. In Figure 7, we show how the workspace encoder attends across adjacent salient times, smoothing out attention and mitigating sensitivity to aliasing effects as seen in its attention weights along with the decoder firing probabilities in Figure 8.

Smoothness in workspace outputs. Towards (2), we show that workspace tokens (i.e. the workspace model outputs) are smooth from their PCA in Figure 9. These illustrate how tokens vary smoothly across time as well as their detected event triggers from the decoder. This smoothness is created by two major factors. First, our saliency pipeline creates smoothed *median* labels as seen in Figure 9 since it has access to the full trajectory while labeling events as opposed to *Keyframe* which maintains a future-unaware labeling pipeline to match test-time labeling. Second, we naturally create smoothness through using an empirical risk minimization optimization across the dataset, creating an averaged encoding that must work robustly across many trajectories. This is seen in Figure 8 where there exist semantic similarities between event frames and a smooth increase in probability over the event occurrence. With these results, we generally advocate for latent memory representations, and claim these characteristics of compressing history are especially significant to ensure downstream imitative policies are robust.

6 Discussion

We present workspace models as a solution to the problem of long-horizon memory in robotic manipulation. By amortizing VLM reasoning at training time, we obtain a fast, robust encoder that generalizes better than VLM-in-the-loop alternatives. The specific instantiation we propose (saliency sets as image

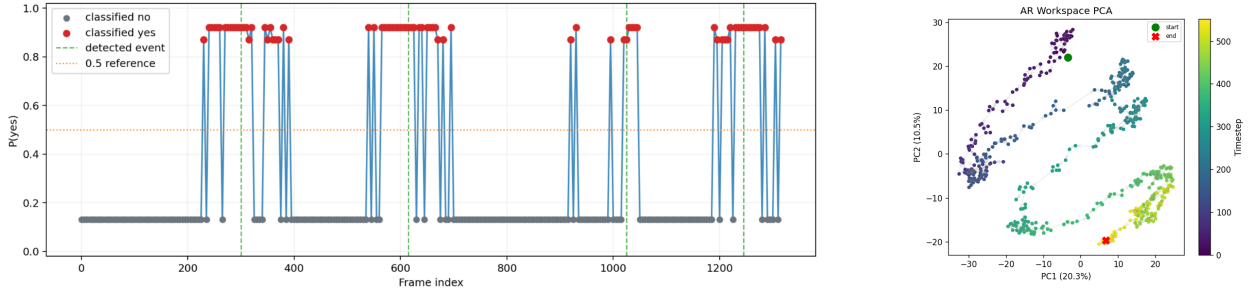


Figure 9: Left: A plot of the VLM detections of an event across time with the green lines representing the labels generated. **Right:** A PCA of the workspace token representations across a rollout.

patches, a DETR-style decoder) is one way to realize this idea, and many alternatives are worth exploring as well as other forms of VLM supervision. For instance, future work could extend the method to multi-camera streams, encode language reasoning traces, or incorporate dynamics prediction.

More broadly, we hope this work encourages that perspective. The core principle extends beyond robotics and memory - anywhere a powerful foundation model is queried repeatedly at test-time to solve a structured subtask, there may be an opportunity to amortize that reasoning into a faster and more robust learned module. And as we have shown, amortization can lead to even stronger results than stronger models in the loop.

Limitations. While the workspace model works well, it relies on a fully autoregressive transformer encoder. This quadratically scales computation, and can become cumbersome at very long sequence lengths. To ameliorate this, we can train workspace models to be recurrent, block diagonal in their attention masks to reduce overhead, or employ KV-caching. Workspace models also receive supervision on what visual information is salient through a VLM which may not be fully grounded in determining which parts of an image are relevant to control. Reinforcement learning can provide an option for grounding in task reward. Lastly, we do not compare to hierarchical methods utilizing language-summaries and VLAs, but this would be interesting to examine along with how workspaces can be combined with generalist policies in the multi-task setting.

7 Acknowledgments

We want to express our gratitude to Younghyo Park, Ryan Bahlous-Boldi, and Antonia Bronars for relevant commentary about the work, along with the Improbable AI Lab community for fruitful discussions. This research was financially supported by the Ministry of Trade, Industry, and Energy (MOTIE), Korea, under the "Global Industrial Technology Cooperation Center program" supervised by the Korea Institute for Advancement of Technology (KIAT). (Grant No. P0028435). This material is based upon work supported by the National Science Foundation Graduate Research Fellowship under Grant No. (NSF grant 2141064).

8 Author Contributions

Nitish Dashora co-developed the project direction, architecture, experimental design, hardware experiments, paper writing, and code development

Douglas Chen contributed to writing, experimental design, and code development

Idan Shenfeld co-developed the project direction, contributed to writing and experimental design, and provided conceptual guidance

John Marangola contributed to hardware experiments

Pulkit Agrawal played a role in paper writing and high-level advising

Max Simchowitz co-developed the project direction, architecture, experimental design, and played a significant role in paper writing

References

- Bernard J. Baars. Global workspace theory of consciousness: toward a cognitive neuroscience of human experience. In Steven Laureys, editor, *The Boundaries of Consciousness: Neurobiology and Neuropathology*, volume 150 of *Progress in Brain Research*, pages 45–53. Elsevier, 2005. doi: [https://doi.org/10.1016/S0079-6123\(05\)50004-9](https://doi.org/10.1016/S0079-6123(05)50004-9). URL <https://www.sciencedirect.com/science/article/pii/S0079612305500049>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.
- Amin Banayeeanzade, Fatemeh Bahrani, Yutai Zhou, and Erdem Biyik. Gabril: Gaze-based regularization for mitigating causal confusion in imitation learning, 2025. URL <https://arxiv.org/abs/2507.19647>.
- Yoshua Bengio. The consciousness prior, 2019. URL <https://arxiv.org/abs/1709.08568>.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Nils Blank, Moritz Reuss, Marcel Rühle, Ömer Erdiñç Yağmurlu, Fabian Wenzel, Oier Mees, and Rudolf Lioutikov. Scaling robot policy learning via zero-shot labeling with foundation models, 2024. URL <https://arxiv.org/abs/2410.17772>.
- Gavin Brown, Adam Pocock, Ming-Jie Zhao, and Mikel Luján. Conditional likelihood maximisation: a unifying framework for information theoretic feature selection. *J. Mach. Learn. Res.*, 13(1):27–66, January 2012. ISSN 1532-4435.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers, 2020. URL <https://arxiv.org/abs/2005.12872>.
- Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.

- Christopher Clark, Yue Yang, Jae Sung Park, Zixian Ma, Jieyu Zhang, Rohun Tripathi, Mohammadreza Salehi, Sangho Lee, Taira Anderson, Winson Han, and Ranjay Krishna. Molmopoint: Better pointing for vlms with grounding tokens, 2026. URL <https://arxiv.org/abs/2603.28069>.
- Pim De Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. *Advances in Neural Information Processing Systems*, 32, 2019.
- Jiafei Duan, Wentao Yuan, Wilbert Pumacay, Yi Ru Wang, Kiana Ehsani, Dieter Fox, and Ranjay Krishna. Manipulate-anything: Automating real-world robots using vision-language models, 2024. URL <https://arxiv.org/abs/2406.18915>.
- Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Re. Cartridges: Lightweight and general-purpose long context representations via self-study, 2025. URL <https://arxiv.org/abs/2506.06266>.
- Product Manual Franka Research 3*. Franka Robotics GmbH, 2025. URL <https://www.franka.de>. Document number: R02210, Release 1.5.1.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, November 2020. ISSN 2522-5839. doi: 10.1038/s42256-020-00257-z. URL <http://dx.doi.org/10.1038/s42256-020-00257-z>.
- Litian Gong, Fatemeh Bahrani, Yutai Zhou, Amin Banayeeanzade, Jiachen Li, and Erdem Biyık. Autofocus-il: Vlm-based saliency maps for data-efficient visual imitation learning without extra human annotations, 2025. URL <https://arxiv.org/abs/2511.18617>.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Sarthak Jain and Byron C. Wallace. Attention is not explanation, 2019. URL <https://arxiv.org/abs/1902.10186>.
- Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artif. Intell.*, 101(1–2):99–134, May 1998. ISSN 0004-3702.
- Jeff Huber Kelly Hong, Anton Troynikov. Context rot: How increasing input tokens impacts llm performance. <https://www.trychroma.com/research/context-rot>, 2025. Accessed: 2025-07-14.
- Oussama Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model, 2024. URL <https://arxiv.org/abs/2406.09246>.
- H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: <https://doi.org/10.1002/nav.3800020109>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109>.

- M. Land, Neil Mennie, and Jennifer Rusted. The roles of vision and eye movements in the control of activities of daily living. *Perception*, 28:1311–28, 02 1999. doi: 10.1068/p2935.
- Anthony Liang, Jesse Thomason, and Erdem Biyik. Visarl: Visual reinforcement learning guided by human saliency, 2024. URL <https://arxiv.org/abs/2403.10940>.
- Huan Liu and Rudy Setiono. Chi2: Feature selection and discretization of numeric attributes. In *Proceedings of the Seventh International Conference on Tools with Artificial Intelligence*, TAI ’95, page 88, USA, 1995. IEEE Computer Society. ISBN 0818673125.
- Wei-Yin Loh. Classification and regression trees. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 1:14 – 23, 01 2011. doi: 10.1002/widm.8.
- Max Sobol Mark, Jacky Liang, Maria Attarian, Chuyuan Fu, Debidatta Dwibedi, Dhruv Shah, and Aviral Kumar. Bpp: Long-context robot imitation learning by focusing on key history frames, 2026. URL <https://arxiv.org/abs/2602.15010>.
- Volodymyr Mnih, Nicolas Heess, Alex Graves, and Koray Kavukcuoglu. Recurrent models of visual attention, 2014. URL <https://arxiv.org/abs/1406.6247>.
- Kaichun Mo, Shilin Zhu, Angel X. Chang, Li Yi, Subarna Tripathi, Leonidas J. Guibas, and Hao Su. Part-Net: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.
- Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer, 2017. URL <https://arxiv.org/abs/1709.07871>.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL <https://arxiv.org/abs/1505.04597>.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 661–668. JMLR Workshop and Conference Proceedings, 2010.
- Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- Akanksha Saran, Ruohan Zhang, Elaine Schaertl Short, and Scott Niekum. Efficiently guiding imitation learning agents with human gaze, 2021. URL <https://arxiv.org/abs/2002.12500>.
- Nur Muhammad Shafiullah, Zichen Cui, Ariuntuya Arty Altanzaya, and Lerrel Pinto. Behavior transformers: Cloning k modes with one stone. *Advances in neural information processing systems*, 35: 22955–22968, 2022.
- Max Simchowitz, Daniel Pfrommer, and Ali Jadbabaie. The pitfalls of imitation learning when actions are continuous. *arXiv preprint arXiv:2503.09722*, 2025.
- Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. Dinov3, 2025. URL <https://arxiv.org/abs/2508.10104>.

- Xingyou Song, Yiding Jiang, Stephen Tu, Yilun Du, and Behnam Neyshabur. Observational overfitting in reinforcement learning, 2019. URL <https://arxiv.org/abs/1912.02975>.
- Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Memer: Scaling up memory for robot control via experience retrieval, 2025. URL <https://arxiv.org/abs/2510.20328>.
- Austin Stone, Ted Xiao, Yao Lu, Keerthana Gopalakrishnan, Kuang-Huei Lee, Quan Vuong, Paul Wohlhart, Sean Kirmani, Brianna Zitkovich, Fei Xia, Chelsea Finn, and Karol Hausman. Open-world object manipulation using pre-trained vision-language models, 2023. URL <https://arxiv.org/abs/2303.00905>.
- Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivi re, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Ga l Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, Andr s Gy r gy, Andr  Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Pluci nska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim P der, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebas-

- tian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 12 2018. ISSN 0035-9246. doi: 10.1111/j.2517-6161.1996.tb02080.x. URL <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>.
- Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle, 2015. URL <https://arxiv.org/abs/1503.02406>.
- Marcel Torne, Andy Tang, Yuejiang Liu, and Chelsea Finn. Learning long-context diffusion policies via past-token prediction, 2025. URL <https://arxiv.org/abs/2505.09561>.
- Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. Mem: Multi-scale embodied memory for vision language action models, 2026. URL <https://arxiv.org/abs/2603.03596>.
- Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories, 2020. URL <https://arxiv.org/abs/2010.14876>.
- Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- Lei Yu and Huan Liu. Feature selection for high-dimensional data: A fast correlation-based filter solution. In *Proceedings of the 20th international conference on machine learning (ICML-03)*, pages 856–863, 2003.
- Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics, 2024. URL <https://arxiv.org/abs/2406.10721>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware, 2023. URL <https://arxiv.org/abs/2304.13705>.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023.

Contents

1	Introduction	2
2	Background: Training History-Based Robotic Policies	3
3	Amortizing History Summarization via Workspace Models	4
3.1	Workspace Model Training	5
3.2	VLM Saliency Labeling	6
4	Workspace Models Achieve High Task Success with Low Inference Latency	7
4.1	Evaluation Tasks: Measuring Different Axes of Memory.	8
4.2	Workspace Models Successfully Amortize VLM Reasoning	8
4.3	Workspace models surprisingly outperform all baselines.	9
5	Why do Workspace Models Exhibit Better Task Performance?	9
5.1	Full Histories and Key-Frame History Both Induce Control-Failure	9
5.2	Keyframe-based methods introduce control aliasing.	10
5.3	Why do Workspace Models Generalize Better?	10
6	Discussion	11
7	Acknowledgments	12
8	Author Contributions	12
A	Extended Related Work	19
B	Architecture and Training Details	20
B.1	Diffusion Policy Details	20
B.2	Workspace Model Details	20
C	Prompting	21
C.1	Event Detection	21
C.2	Post Processing	21
D	Experimental Setup and Details	22
D.1	Simulation	22
D.2	Hardware	23
E	Ablation Studies	24

A Extended Related Work

Generalization in Robotics. We desire a system which can be conditioned on historical information to enable capabilities like in-context learning, adaptation, or memory-intensive reasoning. But in many settings, exposing a learning system to more information can actually hurt performance. Geirhos et al. [2020] describe shortcut learning, the phenomenon where deep learning systems can learn “shortcut” strategies to achieve the training objective that do not generalize to other conditions. This is connected to causal confusion seen in imitation learning [De Haan et al., 2019] or observational overfitting in RL [Song et al., 2019] where access to more information seemingly hurts performance by giving more potential irrelevant cues to learn from, thus hurting generalization. When policies are exposed to observational histories, this confusion can also show up as “copycat” behavior [Wen et al., 2020], where the learner exploits short-range correlations in demonstration trajectories rather than inferring the latent state that should drive action selection. This makes learning policies that require history especially difficult without auxiliary temporal objectives [Torne et al., 2025] or some form of information selection [Mark et al., 2026, Sridhar et al., 2025]. There are a handful of classic approaches to manage information selection.

Information Selection. The idea of information filtering has roots in classic machine learning, statistics, and information theory. Early contributions involved implicit feature selection through the Lasso penalty [Tibshirani, 2018] or explicit statistical testing [Liu and Setiono, 1995]. Other methods use information-theoretic objectives [Loh, 2011, Brown et al., 2012] or correlative measures [Yu and Liu, 2003]. However, these methods often break down in deep learning with high-dimensional data. A popular line of work for studying compression of information and task-relevance is the information bottleneck (IB) design choice [Tishby and Zaslavsky, 2015], where the architecture design forces compression of information in the middle of the neural network. However, models still fall prey to not knowing what may be falsely task-relevant without having enough data [De Haan et al., 2019, Geirhos et al., 2020], which is exacerbated in high-dimensional settings. Moreover, IB lacks a direct analog for modern diffusion-based pipelines. Some newer methods involve learning attention maps, but they do not learn explanations for predictions, and are uncorrelated with feature importance [Jain and Wallace, 2019], incurring quadratic costs. Other methods involve more active selective processing [Mnih et al., 2014] but require expensive RL tuning. So, many practitioners have turned to using human-like general priors for determining what information is useful to maintain for a task. These usually leverage some foundation model, particularly a vision-language model (VLM).

VLM Guidance. One way to leverage a VLM for this is to supervise where the policy should attend. Some work has shown that human gaze can serve as an auxiliary signal for action planning [Land et al., 1999] and robust imitation learning [Saran et al., 2021, Banayeeanzade et al., 2025], but collecting gaze or human-saliency is expensive [Liang et al., 2024]. Therefore, VLMs have been used to provide guidance through different means such as saliency maps [Gong et al., 2025], direct supervision [Blank et al., 2024], or planning [Stone et al., 2023, Duan et al., 2024]. Furthermore, a growing line of work concerns grounding VLMs in order to provide spatial information that can be leveraged by downstream controllers to replace narrowly-trained perception systems [Yuan et al., 2024, Clark et al., 2026]. However, these VLM systems don’t directly address the history compression problem we see when long contexts are required. For this problem, Mark et al. [2026] study using VLMs during execution to select keyframes that correspond to behaviorally salient events. Sridhar et al. [2025] and Torne et al. [2026] use multimodal memory to produce ongoing language summaries to prompt a VLA with. These techniques all require heavy in-the-loop VLM planning, pointing towards latent memory as a potential solution. Our method implements a latent form of memory resembling Cartridges [Eyuboglu et al., 2025], KV-caches distilled from corpora offline to amortize attention for reasoning. Workspace Models similarly distill VLM history curation into a latent embedding for memory-intensive planning.

B Architecture and Training Details

B.1 Diffusion Policy Details

Table 3: Hyperparameters for Diffusion Policy Training.

Component	Hyperparameter	Value
Training	-total-iters	500,000
	-batch-size	128
	-lr	1×10^{-4}
Architecture	-obs-horizon	2
	-act-horizon	8
	-pred-horizon	16
	-diffusion-step-embed-dim	64
	-unet-dims	[64, 128, 256]
	-n-groups	8
	-num-denoising-steps	100
Data	-control-mode	pd_ee_pose

We train imitative policies by approximating the conditional action distribution $p(A_t|O_t)$ through a diffusion model, as done in Diffusion Policy [Chi et al., 2023]. This is done by sampling a true data sample, A_t^0 from $\mathcal{D}$, iteratively corrupting it with noise through a schedule parameterized by α , γ , and σ^2 , and learning a network, ϵ_θ , to predict the corruption noise through the following loss:

$$\mathcal{L}_{\text{action}} = \text{MSE}(\epsilon_k, \epsilon_\theta(A_t^0 + e_k, O_t, k))$$

To sample from this learned distribution, we first sample a noisy $A_t^k \sim \mathcal{N}(0, I)$ and denoise k times to produce an uncorrupted sample A_t^0 . Denoising is done through the following equation:

$$A_t^{k-1} = \alpha(A_t^k - \gamma \epsilon_\theta(A_t^k, O_t, k) + \mathcal{N}(0, \sigma^2 I))$$

Our noise-prediction network, ϵ_θ is implemented as a 1D U-Net with FiLM conditioning [Perez et al., 2017, Ronneberger et al., 2015] where O_t linearly modulates the activations through the U-Net. We append training details for the **Wksp** and **VanillaDP** models in Table 3.

B.2 Workspace Model Details

For training, we use one seed and utilize an 80/20 split. We choose the lowest-validation-loss model and embed the full dataset with that model for imitative training. We now specify model details in this section, particularly how information is fed into the workspace encoder. At every timestep, an image I_t and proprioceptive state x_t are received. We convert I_t into patches through DinoV3 and use a single layer cross-attention pooling mechanism. This converts all patches into 1 token (through a learned fixed query token). We use a simple MLP to map x_t into a proprioceptive token and concatenate it with the pooled image token, giving a tokenized observation function $\bar{o}_t = \text{tokenize}(o_t)$ where $o_t = [I_t, x_t]$. We use a sequence of these tokenized inputs, each with its own padding token, z , to generate a sequence of workspace tokens in one forward pass. The outputs corresponding to the padding inputs are the workspace tokens. During training we use causal masking so that during runtime we do not require the future when producing the current workspace token. In practice, we use a learned positional

embedding within every timestep (across the image, proprioceptive, and padding tokens), and a fixed sinusoidal positional embedding across all timesteps. We include hyperparameters for training our model in Table 4 with abbreviations CD (CUBEDROP), DR (DRAWERRECALL), BB (BALANCEBAR), and HnH (HALFANDHALF). We also shorten `dinov3-convnext-tiny-pretrain-lvd1689m` as DCT and `dinov3-vitb16-pretrain-lvd1689m` as DVB.

C Prompting

C.1 Event Detection

Here, we discuss the details of the VLM labeling pipeline. Recall that the workspace token contains information about both the past and present, **compressing both temporally and spatially**. Therefore, at a given time step, we distinguish between two different types of patches in our salient sets. At every time step, we can have **event** and **transient** patches. Event patches are specialized to an event, are chosen at the time of the event, and remain in the salient set. Transient patches are relevant in the present only and exist in the salient set for the current time.

This leads us to the first set of parameters for the labeling pipelines. We have 4 prompts: `task_description`, `event_detection`, `event_patch`, and `transient_patch`, all working together to label events and patches. We primarily use two models, Qwen3-VL for event labeling, and MolmoPoint-8B for pointing (which is then used for patch labeling). We pass in prompt templates (Fig. 10) alongside the current RGB frame to the labeling models. The event labeling returns either “yes” or “no” for whether an event has been detected. The point labeling returns a list of (`pixel_x`, `pixel_y`) points in the RGB frame’s pixel space, which we normalize to $[0, 1]$ and ultimately convert into patch indices. We also have 2 other shared parameters: `sample_rate`, `max_events`. The sampling rate is the frequency at which we run event labeling. Meanwhile, the max event count controls the maximum number of events; we detail the effect this has on post-processing for each pipeline in Appendix C.2

C.2 Post Processing

For the **Wksp** labeling pipeline, all positive frames are first sorted by their time step. We then group them into segments where a new segment starts only when the gap from the previous positive frame is larger than `min_event_separation_steps`. Then the middle position frame in each segment is chosen as the event. We do a final post processing step: if we have more than `max_events`, we take the first `max_events` events. For **Wksp** we also track transient patches every `tracking_interval` frames.

For the **Keyframe** labeling pipeline, we only start classifying after `ignore_before` and we treat any timestep on a rising edge, where the current classification is “yes” and the previous classification is “no”, as a keyframe. After detecting a keyframe, we skip ahead by `cooldown` and continue sampling every `sample_rate` after as usual. We store the keyframes in a FIFO buffer; when we go over `max_events`, we evict the oldest keyframe and add in the new keyframe.

CubeDrop Prompts

Event Detection Prompt

Is the black gripper above the gray bowl?

Event Point Prompt

gray bowl

Transient Tracking Point Prompt

yellow block

Event Prompt

This is frame {timestep} from a robot demonstration of the task {task_description}. {event_prompt}. Look ONLY at this single frame. Answer with a single word: yes or no.

Point Prompt

Point to the {transient/event patch}

Figure 10: Prompt templates used for event detection and point prediction.

BalanceBar Prompts

Event Detection Prompt

Is the gripper lifting the brown bar straight up from the middle while the bar is still close to the table surface?

Event Point Prompt

brown balance bar

Transient Tracking Point Prompt

brown balance bar

DrawerRecall Prompts

Event Detection Prompt

Is the left robot gripper grasping the yellow block with an open drawer in the scene?

Event Point Prompt

the drawer that the left robot gripper is opening

Transient Tracking Point Prompt

yellow block

HalfAndHalf Prompts

Event Detection Prompt

Is the black robot gripper above the brown box?

Event Point Prompt

robot gripper

Transient Tracking Point Prompt

black cubes

D Experimental Setup and Details

D.1 Simulation

ManiSkill3 is configured to run at 100 Hz. All data collection in simulation is scripted with simple linear motion planners with privileged state information. We run our policies and data collection at 20Hz with PD control on a 7D action including end-effector pose and gripper state.

CUBEDROP In this task, the robot must place exactly 5 cubes into a bowl; however, once something is placed in the bowl, it becomes invisible. We randomly re-spawn blocks after they are placed in the bowl to emphasize the need for present-time perception as well. Once 5 cubes are added, the

robot must press a green button to confirm it is done. The robot succeeds if and only if it presses the green button after having exactly 5 cubes in the bowl. We use 272 trajectories for training. The main failure modes we observe with **VanillaDP** involve not pressing this green button at the right time; however, it does exhibit the strongest manipulation capabilities, always quickly and effectively picking the cubes. This supports the hypothesis regarding how lower observation horizons allow for better function fitting. When adding other frames, **HistoryDP** and **Keyframe** sometimes miss grasps, with **Keyframe** alleviating the fitting issues as the input frames are more consistent.

DRAWERRECALL In this task, the robot observes a secondary robot open a drawer, pick up a cube, place it into the drawer, and then close it. This is done such that it is impossible for the primary robot to know which drawer the cube is within after the drawer is closed. The robot’s task is to then open the correct drawer. We utilize 272 demonstrations for training

BALANCEBAR This task involves lifting a long bar such that it is level (e.g., zero degrees of tilt about the midpoint). However, the bar has one of 3 centers of mass (CoM). The demonstrations begin with an attempted pick from the midpoint, revealing a tilt. The demos then include lifting the bar from the correct CoM, resulting in a balanced lift. For the learner, success is counted if the bar is picked in a balanced way in the second try. We utilize 272 training demonstrations.

D.2 Hardware

We use operational space control **Khatib** [1987] to relay torque commands to the robot.

$$\tau = J^T M_x(q) [K_p e - K_d \dot{e}] + N^T \tau_0$$

Here, e corresponds to pose error, M_x is the task space inertia matrix, and N is a nullspace projector. Our teleoperation data ($N = 300$) is collected at 50Hz with 6D rotation, 3D position, and continuous gripper values. We predict 10D action targets in the same format as our observations. We perform Gram-Schmidt orthonormalization on the 6D rotation component of the predicted actions before they are fed into the operational space controller running at 1kHz.

HALFANDHALF We construct this task where the objective is to equally partition a set of $N \in \{2, 4\}$ 4cm x 4cm x 4cm cubes into two boxes that are too tall to see into with the global camera. As depicted in **Figure 11**, the teleoperator follows a pattern for placing cubes into the bin which necessitates having memory to determine which box to put the next cube inside. We evaluate on the initial configuration shown (i.e. 2 cubes in the middle column). We show results in **Figure 6** for evaluation across 20 trials. We find that **VanillaDP** gets around 50% success since half the time it will put both in the right bin, or do the task correctly. This is since it does not have memory of whether it started with the 4 cube initial state or 2 cube initial state when it is placing the last cube. When we do endow the model with memory in **HistoryDP** and **Keyframe**, we find degradation of performance. The common failure mode was an inability to correctly pick and place the cubes by grasping in between the cubes or not grasping with the right depth. To ameliorate this, we trained these policies with wrist camera feeds observing some performance boost as reported in **Figure 6**. We use 75 demonstrations of

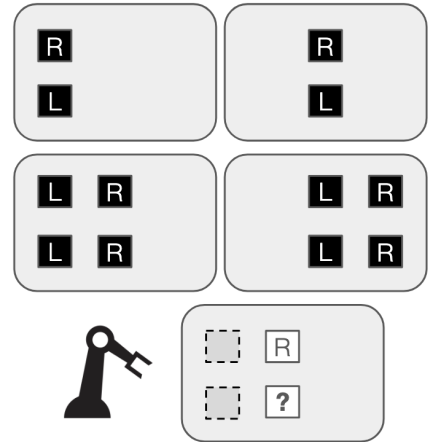


Figure 11: Each black square represents a cube, and the letter represents where the robot must put it (either the right or left box). The test configuration requires memory of the past

Table 6: Success vs. Supervision

Supervision Target	CD	DR	BB	Average
Point	63	100	88	84
Patch	92	100	89	94
Image	0	100	91	64

the 4 configurations shown in Figure 11, resulting in 300 trajectories. Wrist camera feeds were included by simply training a CNN encoder and concatenating it to the global camera encoding. For **Keyframe** and **Wksp**, we pruned erroneous detections of events at $t < 30$ that sometimes appeared in training data. Furthermore, we simply used constant positions for the transient patches (initialized where the cubes start).

E Ablation Studies

Supervision Type We also choose to ablate across different reconstruction targets to see which one forms the best latent representation for the downstream task. Instead of predicting the features for the patch which overlaps where MolmoPoint points, we can directly predict the 2D points during workspace model training. Another option is to predict the features across the whole image rather than the patch of interest. We do this full-image supervision by predicting the average pooled patch feature. Our hypothesis is that point-based supervision is too lossy of a compression which removes relevant details such as rotations (which may be relevant for grasping) or visual features that could be relevant to the downstream task. Full-image supervision, on the other hand, can mix a lot of irrelevant information into the reconstruction target, which can cause potentially spurious correlations or simply increase the noise in our representations. These hypotheses are supported by our results in Table 6 where we see that point-based supervision is strong yet underperforms the patch-based system. Meanwhile, the full-image system performs exceptionally well, but catastrophically breaks down in CUBEDROP, presumably because this task requires the most fine-grained perception since cubes are spawned in randomized locations.

VLM Detection In Figure 9 we include the event firing plot of the VLM for HALFANDHALF that results from using the prompts specified for that task across every frame. It shows when VLM-prompted keyframes are detected and reveals an interesting pattern. First, there is a large amount of noisiness embedded in this detection process, which can be detrimental at runtime for test-time keyframe selection methods. For example, Mark et al. [2026] use “rising-edges” as timesteps to include keyframes, which would result in an inconsistent number of keyframes for every time. While this issue can be softened by higher quality VLM API calls, it requires careful prompt tuning and tuned VLM polling intervals. However, by having access to the full trajectory during train-time, offline saliency-driven labeling enables a more noise-resistant pipeline that is robust to lower quality VLM detection labels and more consistent in timings (since it’s a median). The PCA plot also depicts a smooth representation across time, giving a gradually changing representation across the expert demonstration. In Figure 8, we show the slot probabilities for the trained workspace model on HALFANDHALF as well as the frames at which the slot probabilities cross a threshold. It can be observed that the **Wksp** latent event firing corresponds to highly semantically similar frames (i.e., the gripper dropping a cube in the box) which displays consistency. We hypothesize this consistency is derived from the smoothing that can occur from training on many examples. So, not only does a median-based event time labeling system give resistant labels, but the distillation process also creates a smoothing effect.

Table 4: Hyperparameters for Workspace Training.

Component	Hyperparameter	CD	DR	BB	HnH
Backbone	Model	DCT	DCT	DCT	DVB
	Number of patches	49	49	49	196
	Patch dimension	768	768	768	768
	Register tokens	0	0	0	4
	Frozen	True	True	True	True
Pooler	Pool tokens	1	1	1	1
	Layers	1	1	1	1
	Attention heads	4	4	4	4
	Dropout	0.1	0.1	0.1	0.1
Encoder	Hidden dimension	768	512	512	768
	Layers	2	3	3	2
	Attention heads	4	4	4	4
	MLP ratio	2.0	2.0	2.0	2.0
	Dropout	0.1	0.1	0.1	0.1
	Slots	8	8	8	8
	Workspace tokens	1	1	1	1
Decoder	Layers	2	2	2	2
	Attention heads	4	4	4	4
	MLP ratio	2.0	2.0	2.0	2.0
	Dropout	0.1	0.1	0.1	0.1
	Trunk hidden dim	768	768	768	768
	Trunk layers	2	2	2	2
	Feature dim	768	768	768	768
Loss	Existence loss weight	1.0	0.01	0.01	0.01
	Feature loss weight	1.0	2.0	2.0	1.0
	Existence cost	1.0	1.0	1.0	1.0
	Feature cost	0.5	1.0	1.0	1.0
Training	Batch size	32	24	32	8
	Learning rate	1×10^{-4}	1×10^{-4}	1×10^{-4}	1×10^{-4}
	Weight decay	0.01	0.01	0.01	1×10^{-4}
	Warmup steps	250	250	250	1000
	Max steps	5000	12500	15000	25000
	Gradient clip	1.0	1.0	1.0	1.0

Table 5: Parameters for VLM Prompting.

Component	Hyperparameter	CD	BB	DR	HnH
Event	Model	Qwen3-VL	Qwen3-VL	Qwen3-VL	Qwen3-VL
Transient	Tracking Interval	2	2	2	N/A
Wksp	Sample Rate	5	4	5	5
	Max Events	5	1	1	4
	Min Event Separation	20	16	15	75
Keyframe	Sample Rate	5	2	15	5
	Cooldown	30	96	1300	300
	Ignore Before	0	22	100	50